EPSR-test-ex37 makelattice: crystal from a unit-cell file

Program(s): makelattice | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises `makelattice` [1], which builds a crystal of tethered one-atom molecules by repeating a unit cell `na` x `nb` x `nc` times — the starting point for EPSR models of crystalline or amorphous network materials [2]. Three unit cells cover fractional and absolute coordinates, a cubic cell, a cell with gamma = 120° and a triclinic cell, and the optional lines of the cell file.

Code exercised

Fortran (original, fortran_src/):

- `makelattice.f`: `makelattice` (cell file, replication, tethering)
- `auxiliary_routines.f`: `convertalphabetagamma` (cell angles to box vectors), `initran1`
- `ato_manipulation_rev4.f`: `ctype`, `ftype`; `ato_read_and_save.f`: `moloutc`

Rust (port, rust/epsr-core/src/):

- `tools/makelattice.rs`: `run`, `convertalphabetagamma`, `initran1`
- `ato.rs`: `ctype`, `ftype`, `moloutc`, `cosd_intrinsic/sind_intrinsic`
- `rng.rs`: `Ran1`

Physical system

Three test crystals at 300 K. Rock salt NaCl (a = 5.64 Å, 8 atoms per cubic cell in fractional coordinates, Na: epsilon 0.5443 kJ/mol, sigma 2.35 Å, q +1; Cl: 0.4184 kJ/mol, 4.4 Å, q -1; `ecore dcore` 0.5 2.0; stored random-number state) as a 2×2×2 supercell of 64 atoms, box 11.28 Å. A triclinic Si/O cell (4.1, 4.6, 5.3 Å; 80°, 100°, 110°; absolute coordinates; 3 atoms) as 2×2×1 (12 atoms). An argon cell (3, 4, 5 Å; gamma 120°; 2 atoms) as 3×2×1 (12 atoms).

Inputs

Unit-cell files written for this test: `NaCl.cell`, `tric.cell` (atom-type line without `iexchange`, no ecore line, no random-number line: generator reset) and `ortho.cell` (no coordinate-type line, count read from line 3). Commands: `makelattice NaCl.cell 2 2 2 < ml1.stdin` (output name with extension), `makelattice < ml2.stdin` (file and `na` on one line, `nb nc` prompted), `makelattice < ml3.stdin` (all prompted).

How the accuracy was verified

`commands.txt` (3 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.

Three program calls; compared outputs `NaCl_222.ato`, `tric.ato`, `ortho.ato`, all identical. The triclinic path is active: the reference `tric.ato` holds box lengths 8.2, 9.2, 5.3 Å and the converted angles 110°, 12.2387°, 145.0° from `convertalphabetagamma`.

Notes

Fortran details reproduced: the box is always written as non-cubic (`cubic` is never set); `cosd`/`sind` are the gfortran intrinsics (not EPSR's own functions, since they are not declared `external`); `natomsph` is implicitly INTEGER, so `ctype` stores the bit pattern of integer 1 (1.4E-45) as `natomsphere`.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).

[2] A. K. Soper, "Partial structure factors from disordered materials diffraction data: An approach using empirical potential structure refinement", Phys. Rev. B 72, 104204 (2005).

[3] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.